

QP-2.1 Linguagens de Programação: Questões pré-classe para a aula 1 de "Sintaxe e Semântica"

Total de pontos 17/17

Essas questões devem ser respondidas até 2 horas antes da respectiva aula na qual os conteúdos serão tratados, após assistir/ler o material deixado para aula.

E-mail *

ramonsoares@discente.ufcat.edu.br

0 de 0 pontos

Nome *

Para registro das respostas

RAMON SOARES MENDES DE MENESES LEITE

2 de 2 pontos

QP-1.1: A sintaxe * 1/1

- ☐ Define o modelo de execução de um programa para uma certa linguagem.
- ☐ Define o comportamento das unidades de um programa.
- ☐ Define a legibilidade de uma linguagem de programação.
- ☐ Define a confiabilidade de uma linguagem de programação.
- ☐ Define o paradigma da linguagem de programação.
- ☒ Define a estrutura de todas as unidades de um programa.

QP-1.2: A semântica * 1/1

- ☐ Define a confiabilidade de uma linguagem de programação.
- ☐ Define a legibilidade de uma linguagem de programação.
- ☐ Define a estrutura de todas as unidades de um programa.
- ☐ Define o modelo de execução de um programa para uma certa linguagem.
- ☐ Define o paradigma da linguagem de programação.
- ☒ Define o comportamento das unidades de um programa.

2 de 2 pontos

Considere as especificações/definições abaixo.

- Os caracteres que são reconhecidos como identificadores válidos.
- O efeito de se comparar duas strings usando o operador ==.
- Que um bloco de comandos (exemplo: dentro de um for) é especificado por {.
- O modelo de implementação da linguagem (compilada, interpretada, híbrida)
- O valor inicial de uma variável não inicializada pelo programador.
- O que é realizado quando o operador >= é aplicado a uma variável.
- Como comentários são definidos e reconhecidos em um programa.
- Como caracteres de escape (por exemplo \n para nova linha) são especificados dentro de uma string.
- Número de bytes de um tipo inteiro.
- As palavras reservadas da linguagem.

QP-1.3: Quais dessas especificações (de 1 a 10) poderiam fazer parte da SINTAXE de uma linguagem? * 1/1

- ☐ 2, 5, 6, 9
- ☐ 2, 4, 5, 6, 9
- ☐ 1, 3, 7, 10
- ☐ 1, 3, 5, 7, 10
- ☐ 2, 5, 6, 9, 10
- ☒ 1, 3, 7, 8, 10
- ☐ 2, 5, 6, 9, 10
- ☐ 1, 3
- ☐ 2, 6
- ☐ 1, 3, 5, 7, 9, 10

QP-1.4: Quais dessas especificações (de 1 a 10) poderiam fazer parte da SEMÂNTICA de uma linguagem? * 1/1

- ☐ 1, 3, 7, 8, 10
- ☐ 1, 3, 7, 10
- ☐ 2, 4, 5, 6, 9
- ☐ 2, 5, 6, 9, 10
- ☐ 1, 3
- ☒ 2, 5, 6, 9
- ☐ 1, 3, 5, 7, 9, 10
- ☐ 1, 3, 5, 7, 10
- ☐ 2, 5, 6, 9, 10
- ☐ 2, 6

1 de 1 pontos

Considere o seguinte trecho de código C:

```
int calc(int a, double b) {
    int tmp = (int) b;
    if (tmp == 0) { return 0; }
    return tmp * a;
}
```

QP-1.6: Assinale a alternativa correta: * 1/1

- ☒ tmp é um token identificador
- ☐ tmp é um lexema identificador
- ☐ não é token e nem lexema.
- ☐ "int a" é um token declaração
- ☐ = é um token operador
- ☐ return é um token identificador
- ☐ return é um lexema identificador

6 de 6 pontos

Considere a seguinte regra BNF

```
<statement> -> <varlist> = <exprlist>
              | do <block> end
              | while <exp> do <block> end
```

Para a regra BNF, são exemplos de símbolos terminais * 1/1

- ☒ do, while
- ☐ do <block>
- ☐ -, =
- ☐ do <block> end
- ☐ apenas o end

Para a regra BNF, são exemplos de símbolos não-terminais * 1/1

- ☐ do <block> end
- ☐ <varlist> = <exprlist>
- ☒ <block>
- ☐ do, while
- ☐ apenas o <statement>
- ☐ -, =
- ☐ apenas o end
- ☐ do <block>

Qual a diferença entre um símbolo terminal e um símbolo não terminal? 1/1

- ☐ Um símbolo terminal representa uma palavra reservada da linguagem, enquanto um símbolo não-terminal representa símbolos que podem variar no código (como nomes de variáveis).
- ☒ Um símbolo terminal representa um lexema válido para um código naquela linguagem, enquanto um símbolo não-terminal representa uma abstração que pode ser expandida (ou substituída) várias vezes até produzir diversos lexemas possíveis.
- ☐ Um símbolo terminal sempre fica à esquerda de uma regra BNF, enquanto que o não-terminal fica à direita.
- ☐ Um símbolo não-terminal sempre fica à esquerda de uma regra BNF, enquanto que o terminal fica à direita.
- ☐ Um símbolo não-terminal representa um lexema válido para um código naquela linguagem, enquanto um símbolo terminal representa uma abstração que pode ser expandida (ou substituída) várias vezes até produzir diversos lexemas possíveis.
- ☐ Um símbolo não-terminal representa uma palavra reservada da linguagem, enquanto um símbolo terminal representa símbolos que podem variar no código (como nomes de variáveis).

O lado esquerdo (LHS) de uma regra BNF representa: * 1/1

- ☐ Um símbolo terminal resultado da atribuição especificada.
- ☒ Um símbolo não-terminal sendo definido.
- ☐ Um símbolo não-terminal resultado da atribuição especificada.
- ☐ Um símbolo terminal sendo definido.

O lado direito (RHS) de uma regra BNF representa: * 1/1

- ☐ Um símbolo terminal resultado da atribuição especificada.
- ☐ Um símbolo terminal sendo definido.
- ☐ Um símbolo não-terminal resultado da atribuição especificada.
- ☒ Uma ou várias possíveis expansões de um símbolo não-terminal.
- ☐ Um símbolo não-terminal sendo definido.
- ☐ Uma ou várias possíveis expansões de um símbolo terminal.

Considere a seguinte regra BNF: <f> -> if <expr> then <stmt> else <stmt>. Quais dos seguintes códigos estão de acordo com essas regras BNF, supondo que (a < 0) satisfaz a regra de <expr> e tanto a++; e b++; satisfazem a regra <stmt>? * 1/1

- ☒ if (a < 0) then a++; else b++;
- ☐ if (a < 0) then b++;
- ☐ if (a < 0) then a++;
- ☐ Todas as demais alternativas estão corretas.

1 de 1 pontos

Considere a seguinte regras BNF:

```
<x> -> a|b|c
<a> -> a|a|a|a
<b> -> b|b|b|b
```

Quais dos seguintes códigos estão de acordo com essas regras BNF? * 1/1

- ☐ "ab" e ""
- ☐ "ababababababab"
- ☒ "aaaaabbbb" e "aab"
- ☐ "aba"
- ☐ "aaaaabbbb" e "bbaa"

3 de 3 pontos

Sua linguagem precisa permitir atribuições declarações de variáveis como C, podendo indicar tanto o tipo de uma variável como uma sequência de variáveis com o mesmo tipo como nas duas sentenças abaixo:

```
int x;
int a, b, c, d;
```

Considere os grupos de regras BNF abaixo para <decl_variavel>.

(A) <decl_variavel> -> int <ident>, <decl_variavel>;

(B) <decl_variavel> -> int <ident>; | int <ident>, <decl_variavel>

(C) <decl_variavel> -> int <idents>;

<idents> -> <ident> | <idents>, <ident>

(D) <decl_variavel> -> int <idents>; | int <ident>, <idents>; | int <idents>, <idents>, <idents>; | int <idents>, <idents>, <idents>, <idents>;

(E) <decl_variavel> -> int <idents>;

<idents> -> <decl_variavel>, <idents>; | <idents>, <idents>

Qual dos grupos de regras BNF é capaz de descrever a construções pretendidas para a linguagem? * 1/1

- ☒ C
- ☐ E
- ☐ B
- ☐ A
- ☐ D

O símbolo de start de uma gramática BNF indica * 1/1

- ☐ Qual regra é a primeira na ordem de avaliação das regras BNF.
- ☐ Qual regra será aplicada para a primeira linha de um programa da linguagem.
- ☒ Qual regra deverá ser necessariamente casada com todo um programa da linguagem.
- ☐ Qual regra tem a prioridade sobre avaliação das regras BNF, quando mais de uma se aplica.

Uma derivação termina quando * 1/1

- ☒ Todos os símbolos produzidos na derivação são terminais.
- ☐ É avaliada a última linha do programa.
- ☐ Todos os símbolos não-terminais da primeira linha do programa são identificados.
- ☐ Os símbolos produzidos na derivação NÃO possuem o símbolo de start.
- ☐ Os símbolos produzidos na derivação possuem o símbolo de start.

1 de 1 pontos

Untitled section

Considere a seguinte gramática

```
<attr> -> <id> := <expr>
<id> -> A | B | C
<expr> -> <expr> + <expr>
        | <expr> * <expr>
        | ( <expr> )
        | <id>
```

Considere as seguintes possíveis derivações para A := B + B + B.

DERIVAÇÃO 1

```
<attr> -> <id> := <expr>
<attr> -> A := <expr>
<attr> -> A := B + <expr>
<attr> -> A := B + B + <expr>
<attr> -> A := B + B + B
```

DERIVAÇÃO 2

```
<attr> -> <id> := <expr>
<attr> -> A := <expr>
<attr> -> A := <expr> + <expr>
<attr> -> A := <expr> + <expr> + <expr>
<attr> -> A := <id> + <expr> + <expr>
<attr> -> A := <id> + <id> + <id>
<attr> -> A := B + B + B + <id>
<attr> -> A := B + B + B
```

DERIVAÇÃO 3

```
<attr> -> <id> := <expr>
<attr> -> A := <expr>
<attr> -> A := <expr> + <expr>
<attr> -> A := <id> + <expr>
<attr> -> A := B + <expr>
<attr> -> A := B + <expr> + <expr>
<attr> -> A := B + <id> + <id>
<attr> -> A := B + B + B
```

DERIVAÇÃO 4

```
<attr> -> <id> := <expr>
<attr> -> A := <expr>
<attr> -> A := <expr> + <expr>
<attr> -> A := <expr> + <id>
<attr> -> A := <expr> + B
<attr> -> A := <expr> + <expr> + B
<attr> -> A := <expr> + <id> + B
<attr> -> A := <expr> + B + B
<attr> -> A := <id> + B + B
<attr> -> A := B + B + B
```

Qual das derivações representa a derivação à esquerda para A := B + B + B? 1/1

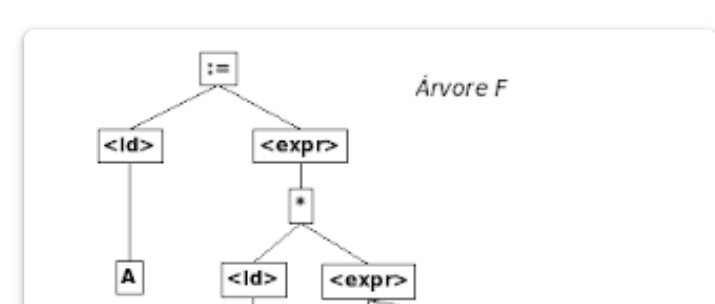
- ☒ 3
- ☐ 4
- ☐ 2
- ☐ 1

1 de 1 pontos

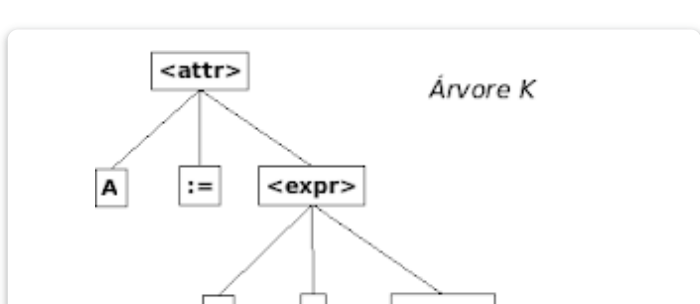
Considere as regras BNF

```
<attr> -> <id> := <expr>
<id> -> A | B | C
<expr> -> <expr> + <expr>
        | <expr> * <expr>
        | ( <expr> )
        | <id>
```

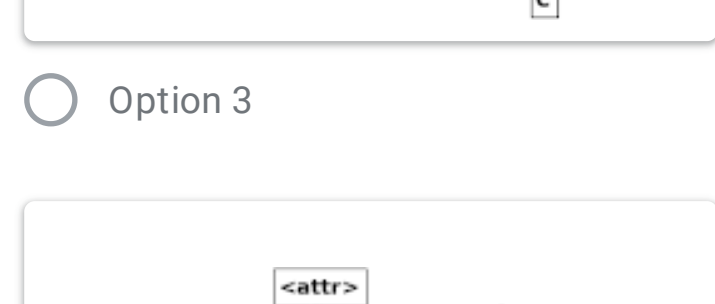
Para o código A := B * (A + C), qual das árvores de análise K, J, F, T corresponde à derivação da sentença? * 1/1



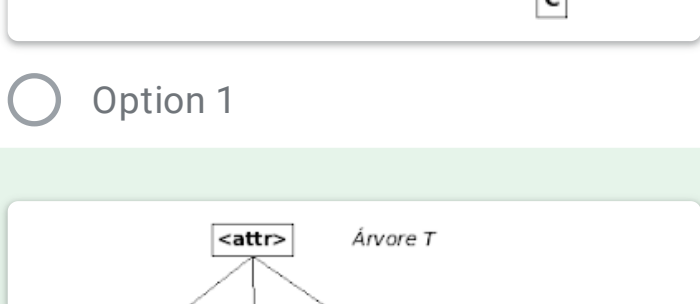
Option 3



Option 1



Option 2



Option 4